

HealthCentral

Technical Companion

How agentic frameworks encode software-engineering discipline to keep LLMs on task

A deep dive on superpowers, GSD v1/v2, and everything-claude-code

CS4610 Senior Project (Par 1) — Danny Tran

Cal Poly Pomona — May 15th, 2026

Table of Contents

Table of Contents..... 2

1. Why this companion exists..... 3

2. Why LLMs drift, and why SWE structure is the fix 3

3. The SDLC → agentic-scaffold mapping 3

4. everything-claude-code: the kitchen-sink harness..... 5

 4.1 Architecture 5

 4.2 Agent roster (selected) 5

 4.3 Hooks as enforcement..... 6

 4.4 AgentShield 6

 4.5 Concrete pipeline: "add JWT authentication" 6

 4.6 How HealthCentral used ECC 7

5. GSD: from prompts to orchestrator 7

 5.1 GSD v1 — prompts in ~/.claude/commands/ 7

 5.2 GSD v2 — a file-driven orchestrator 7

 5.3 The agent roster 8

 5.4 "Hit Ratio" and related metrics 8

 5.5 Concrete pipeline: building a crossword game..... 9

 5.6 How HealthCentral used GSD v2 9

6. Superpowers: the skill-based discipline..... 9

 6.1 The Iron Law10

 6.2 The pipeline.....10

 6.3 Git worktree isolation10

 6.4 Concrete pipeline: a todo list app.....11

 6.5 How HealthCentral used superpowers.....11

7. Comparing the three frameworks11

8. Is this agile, waterfall, or something new?13

9. Failure modes and how each framework answered them.....13

10. Reflections: rigid structure beats clever prompting14

11. References.....14

1. Why this companion exists

The main senior-project report traces what happened across four months: from vibe coding to autonomous harnesses, with HealthCentral as the concrete artifact. This companion goes one layer deeper. It answers the question the main report poses but does not fully unpack: if agentic tools have crossed a threshold, what exactly is inside the harness that made that possible?

The thesis of this companion is simple. Large language models, left to themselves, drift. They lose the thread of the original task, forget constraints as context fills, over-help by substituting missing entities, and pattern-match themselves into hallucinations. Engineers responded by re-learning an old lesson: software engineering is not mostly about typing code. It is about requirements, design, review, and disciplined feedback loops. The frameworks that actually keep agents on task — superpowers, GSD (v1 and v2), and everything-claude-code — do it by encoding decades of software-engineering practice as scaffolding the model cannot skip.

This companion dissects that encoding for each of the three frameworks, shows the concrete pipelines and tool calls, and maps each stage back to the traditional software development lifecycle (SDLC). The mapping is the argument.

2. Why LLMs drift, and why SWE structure is the fix

The most important empirical result in this area is the goal-drift paper: every frontier model evaluated — including state of the art — exhibits systematic goal drift when exposed to competing objectives, long-horizon tasks, or extended instrumental goal pursuit. In one experiment an agent tasked with maximizing profits drifted into price-fixing and dishonesty after about a hundred turns. The mechanism is well understood: attention mathematically favors recent tokens, so older constraints decay; pattern matching takes over as context fills; and mistakes compound when the agent modifies state and adapts to it.

Anthropic's own alignment auditing work identifies four recurring archetypes: premature action without grounding, over-helpful substitution of missing entities, context pollution from distractors, and fragile execution under load. Every one of these is a problem an experienced senior engineer recognizes instantly, because real software-engineering practice evolved to prevent them in human teams. Requirements documents prevent premature action. Design reviews prevent over-helpful improvisation. Code reviews catch hallucinated context. Test suites catch fragile execution.

The move that makes agentic tools work is deceptively simple: take those practices and make them structural properties of the harness, not suggestions in a prompt. A prompt can be forgotten. A hook cannot. A suggestion can be ignored. A gate that refuses to let the agent advance cannot. The three frameworks we deep-dive below are three flavors of the same move.

3. The SDLC → agentic-scaffold mapping

Below is the backbone of the argument. Each traditional SDLC stage maps to a concrete artifact, agent role, or hook inside at least one of the three frameworks we study. The mapping is complete enough that a senior developer reading it would recognize their own workflow, rewritten as code.

SDLC Stage	Traditional role	Agentic scaffold	Where it lives in the frameworks
Requirements	Analyst / PM gathers needs	Spec-first PROJECT.md / REQUIREMENTS.md	GSD v2 Researcher agents (x4 parallel); superpowers brainstorming skill; everything-claude-code planner
Design	Architect drafts design doc	Design skill writes docs/specs/YYYY-MM-DD-design.md before code	superpowers brainstorming + writing-plans skills; ECC architect agent; GSD Planner
Design review	Arch review meeting	Plan Checker / ADR validator blocks execution on incomplete plan	GSD Plan Checker agent; superpowers spec self-review; ECC architect handoff
Implementation	Developer codes	Implementer subagent in fresh 200K context window	GSD Worker (parallel waves); superpowers subagent-driven-development; ECC tdd-guide
TDD (RED)	Dev writes failing test	Enforced: test file before src file	superpowers Iron Law in tdd skill; ECC tdd-guide enforces RED->GREEN; GSD "truths" before "artifacts"
Code review	Senior engineer reviews diff	Cross-model adversarial review gate	superpowers spec-reviewer + code-reviewer; ECC code-reviewer + language-specific reviewers; GSD Verifier
Security review	AppSec scans	Static rules + LLM-as-red-team	ECC AgentShield (102 rules) + --opus red-team mode; superpowers hooks optional
Integration	CI, merge to main	Milestone reconciliation + squash-merge	GSD milestone auto-advance; superpowers finishing-a-development-branch; ECC PR workflow

SDLC Stage	Traditional role	Agentic scaffold	Where it lives in the frameworks
HITL gate	Release manager approves	Orchestrator asks user before destructive or out-of-spec actions	All three pause at spec approval; GSD stuck-loop escalates to human; ECC PreToolUse hooks
Retrospective	Sprint retro	DECISIONS.md updates; pattern extraction on Stop hook	GSD DECISIONS.md; ECC Stop hook extracts reusable skills

Three observations about this table matter for the rest of the report. First, the stages are not optional. A framework that lets the agent skip “design review” produces the same drift every human team suffers without design review. Second, the review step is usually a different agent — often a different vendor’s model — because diversity of perspective is itself a SWE pattern (peer review, change advisory boards). Third, HITL gates survive because some decisions are irreversible: destructive writes, merges to main, production deploys. The frameworks differ in where they place those gates, not whether.

4. everything-claude-code: the kitchen-sink harness

everything-claude-code (ECC) by Affaan Mustafa began as a hackathon project and matured into a production configuration that spans Claude Code, Cursor, Codex, and OpenCode from a single repository. It is the densest of the three frameworks: many dozens of specialized subagents, many dozens of skills, a dedicated security scanner called AgentShield, a companion GitHub App (ECC Tools), and a rich hook layer that enforces quality gates at every tool-execution boundary.

4.1 Architecture

The system is organized around four artifact types that together express the entire SDLC as runnable configuration: agents (specialists), skills (procedures), rules (conventions), and hooks (enforcement). The single source of truth is an AGENTS.md file that every platform reads; the platform-specific adapters (.claude/, .cursor/, .codex/, .opencode/) translate it into the primitives each host supports.

4.2 Agent roster (selected)

ECC ships a layered roster. The top layer is orchestration: planner (decomposes features into tasks), architect (design and API contracts), chief-of-staff (multi-agent coordination). The second layer is quality: code-reviewer (general), security-reviewer (vulnerabilities), build-error-resolver. The third layer is language-specific reviewers and build-resolvers for TypeScript, Python, Go, Java, Rust, C++, Kotlin, C#, Dart, and Swift. The fourth layer is specialists: tdd-guide, e2e-runner (Playwright), database-reviewer, a11y-architect (WCAG 2.1 AA), refactor-cleaner,

performance-optimizer, silent-failure-hunter, and so on. Numbers vary between releases — the repo roster has grown past the README’s original “48 agents” claim — but the structure is stable.

4.3 Hooks as enforcement

This is where ECC differs most sharply from superpowers. Where superpowers prefers skill-based mandatory invocation, ECC leans on Claude Code’s hook system to make certain properties unskippable. Hook profiles (minimal, standard, strict) are selectable per session via environment variables.

- **SessionStart:** loads persistent context from prior sessions (learned patterns, TODOs, git state).
- **PreToolUse:** secret-detector, dev-server-blocker (blocks npm run dev outside tmux), git-push-reminder, pre-commit-quality-check, doc-file-warning, strategic-compact-recommender.
- **PostToolUse:** prettier-formatter, typescript-checker (runs tsc --noEmit), console-log-warning, build-analysis, pr-logger, design-quality-check.
- **Stop:** extracts reusable skills from the session transcript, records tokens and cost, suggests context compaction for the next run.

4.4 AgentShield

AgentShield is ECC’s security layer, also available as a standalone scanner. It runs roughly 102 static rules across five categories: secrets (AWS, GitHub, Stripe, OpenAI key patterns), permission auditing (tool-access breadth), hook-injection analysis, MCP server risk profiling, and agent-config review (prompt injection, instruction overlap). In advanced mode, a three-agent red-team pipeline runs on Claude Opus: an attacker searches for exploits, a defender evaluates protections, and an auditor synthesizes the risk report. Output formats include terminal, JSON, Markdown, and a self-contained HTML dashboard.

4.5 Concrete pipeline: "add JWT authentication"

The following is a faithful reproduction of the pipeline ECC runs for a real feature, adapted from the repo’s documentation and walkthrough posts:

1. SessionStart hook loads the user’s prior context and the repo’s AGENTS.md into the session.
2. User types `/ecc:plan "add JWT authentication with refresh rotation"`.
3. planner subagent decomposes the feature into: (a) token schema, (b) generation endpoint, (c) middleware auth, (d) refresh / logout.
4. User types `/tdd`. tdd-guide enters the RED phase: failing tests for `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, and `GET /me` (authenticated).
5. Implementation follows; PostToolUse runs `tsc --noEmit` and `prettier` on every edit. typescript-reviewer flags any implicit any or improperly typed Promise.

6. User types `/code-review`. `code-reviewer` evaluates quality; `security-reviewer` scans for token leakage and missing rate limiting; `AgentShield` runs the 102-rule scan.
7. User addresses feedback (add `express-rate-limit`). Tests re-run; coverage stays above threshold.
8. User types `/e2e-test auth flow`. `e2e-runner` drives Playwright through `register` → `login` → `/me` → `logout` → `/me` (expected 401).
9. Pre-commit hook validates conventional commit message and absence of secrets in diff. `git-push-reminder` asks the user to confirm the diff before pushing.
10. Stop hook extracts the reusable pattern “JWT authentication with refresh rotation” into `~/.claude/patterns/` and records token spend for the session.

4.6 How HealthCentral used ECC

HealthCentral adopted three parts of ECC explicitly. First, `AgentShield` was wired as a `PreToolUse` hook scanning every edit for PHI patterns; any tool call that would emit identifiable medical data onto a network interface is blocked before it runs. Second, the `tdd-guide` and language-specific reviewers were used during the RAG pipeline build, where correctness matters more than speed. Third, the Stop hook pattern-extraction saved the project’s own `SKILL.md` files for analyte-synonym mapping and unit normalization, so the second patient’s PDFs were processed with the first patient’s corrections already learned.

5. GSD: from prompts to orchestrator

5.1 GSD v1 — prompts in `~/.claude/commands/`

GSD v1 was a collection of Markdown prompt templates dropped into `~/.claude/commands/`. The slash commands it registered — `/gsd-new-project`, `/gsd-discuss-phase`, `/gsd-plan-phase`, `/gsd-execute-phase`, `/gsd-verify-work`, `/gsd-ship`, `/gsd-next` — read like the phases of a waterfall project, because that is what they encoded. The thesis was correct: structure the conversation into phases so the model cannot conflate requirements with implementation. The limitation was that nothing enforced the structure. The model could — and often did — skip phases, drift across phase boundaries, or re-implement the same piece three times as context filled. v1 was a prompt-level fix to a harness-level problem.

5.2 GSD v2 — a file-driven orchestrator

GSD v2 fixes the architecture. It is a standalone TypeScript CLI built on the Pi SDK that runs as its own process and has direct access to the agent harness. Installation is a single `npm` invocation; configuration files land in `~/.claude/skills/`, `~/.codex/skills/`, or `~/.config/opencode/` depending on the host.

The core design pattern is a file-driven state machine. Five markdown files in `.gsd/` constitute the project’s durable brain:

- **PROJECT.md** — vision, success criteria, non-negotiable constraints.

- **REQUIREMENTS.md** — scoped requirements with phase traceability.
- **ROADMAP.md** — the milestone → slice → task hierarchy with dependencies.
- **DECISIONS.md** — architecture decisions and rationale (an ADR log).
- **STATE.md** — current blockers, in-progress tasks, runtime state.

5.3 The agent roster

GSD v2 implements a thin-orchestrator pattern. A coordinator reads state, determines the next work unit, and spawns specialists, each in a fresh 200K-token context window so no one agent drowns in history.

- **Scout** — fast codebase reconnaissance; returns compressed handoff summaries.
- **Researcher (x4 parallel)** — investigates stack, feature landscape, pitfalls, and external intelligence, one dimension per agent.
- **Planner** — converts research into a task specification with explicit "truths" (automated checks) and "artifacts" (files that must exist).
- **Plan Checker** — before any execution, validates the plan. Every truth must have an artifact; every artifact must have a path to a truth. Incomplete plans are rejected.
- **Worker (and JS-Pro / TS-Pro specialists)** — executes tasks in dependency-aware parallel waves.
- **Verifier** — runs truths automatically and judges artifacts with code inspection. Gates merge.
- **Debugger** — diagnoses failures when Verifier rejects a task; loops with Worker until fixed or escalation.

5.4 "Hit Ratio" and related metrics

The community sometimes discusses GSD v2 in terms of a Hit Ratio. A careful search of the official GSD v2 documentation did not surface a formal definition for a metric by that name, so this companion treats "Hit Ratio" as a community shorthand and explains the concrete metrics the orchestrator actually tracks, which together give you the same intuition:

- **Task completion rate** — tasks completed divided by tasks planned for the current slice.
- **Verification pass rate** — artifacts that pass the Verifier on first attempt divided by artifacts produced. This is the closest analog to "hit ratio" in the everyday sense: how often does the model's first output clear review?
- **Token and cost accounting** — cumulative tokens and cost per milestone, exposed in STATE.md and cost dashboards.
- **Stuck-loop detection** — a sliding-window pattern analyzer over recent dispatch history looks for cyclic sequences ($A \rightarrow B \rightarrow A \rightarrow B$ or $A \rightarrow A \rightarrow A$). On detection, GSD retries once with a deep-diagnostic prompt, then halts auto-mode and flags the exact file it was stuck on, escalating to the human.

If you are building your own version of this metric, verification pass rate is the one that matters most: it is a direct, machine-checkable measure of how often the planner + worker loop produces correct output without rework. It degrades when plans are under-specified, when tasks are too large, or when the Worker's context is polluted. Tracking it over time tells you whether your specifications are improving.

5.5 Concrete pipeline: building a crossword game

Lex Christopherson's demo of GSD v2 building a crossword game is a useful concrete reference. The pipeline is:

11. User runs `/gsd-new-project "crossword game, web, plays a daily NYT-style puzzle"`.
12. Orchestrator spawns four Researchers in parallel: stack recon (React / Vite / Jest already in scope), feature landscape (daily puzzle source, hints, timer), pitfalls (crossword symmetry, clue-disambiguation), external intelligence (puzzle APIs, open datasets).
13. Researchers write findings to `.planning/research/*.md`. Orchestrator synthesizes them into `REQUIREMENTS.md` and an initial `ROADMAP.md`.
14. Planner writes the first slice: `"render grid from a puzzle JSON"`. Plan Checker validates every truth has a test file and every artifact has a source file.
15. Worker executes the slice in dependency-aware waves: data model → grid component → cell selection. Each wave commits atomically.
16. Verifier runs the test suite, checks the rendered output, and judges code quality. Passes.
17. Orchestrator advances to the next slice: `"keyboard input and word highlighting"`. Loop.
18. Periodically, context accumulates in the main session; the orchestrator clears it using `auto-compact` and resumes from `STATE.md` without losing progress.
19. After all slices in a milestone pass verification, milestone reconciliation re-reads `ROADMAP.md` and confirms every success criterion is met before squash-merging to main.

5.6 How HealthCentral used GSD v2

HealthCentral used GSD v2 for the RAG evaluation campaign. The research agents pulled recent papers on medical-report RAG, chunking strategies for lab data, and citation-grounding techniques. The Planner decomposed the evaluation into parameter-sweep slices (chunk size, embedding model, retrieval threshold, reranker on/off). Workers ran each slice in parallel against a locked held-out set of patient-question/answer pairs. The Verifier checked that every configuration produced a structured report with recall-at-k, citation-fidelity, and a qualitative rating, and the Orchestrator advanced automatically until the Pareto frontier was characterized. Overnight, with stuck-loop detection preventing runaway cost, the campaign completed without human babysitting.

6. Superpowers: the skill-based discipline

Superpowers, authored by Jesse Vincent (obra/superpowers), takes a different approach to the same problem. Where ECC leans on hooks and GSD leans on a programmatic orchestrator, superpowers leans on skills — a set of mandatory procedural skill files whose invocation is structurally required before the agent can proceed. The enforcement lives in the skill text and the plugin manifest, not in PreToolUse hooks.

6.1 The Iron Law

Superpowers' signature constraint is the Iron Law: "NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST." It is enforced by the test-driven-development skill, which mandates the classical RED → GREEN → REFACTOR cycle and forbids the agent from adding features, refactoring unrelated code, or "improving" beyond what the current test requires. The same law is applied recursively when a developer writes a new skill for superpowers itself: you have to show an agent failing without the skill, demonstrate that the skill fixes the failure, and only then refactor.

6.2 The pipeline

The standard superpowers workflow proceeds through five phases, each gated by an approval step:

20. Brainstorming. The brainstorming skill is mandatory. The agent reads project context, asks clarifying questions one at a time, proposes two to three design alternatives with trade-offs, presents the chosen design section by section (architecture → components → data flow → error handling) with user approval at each section, and finally writes the full design doc to docs/superpowers/specs/YYYY-MM-DD-topic-design.md. The agent then runs a self-review looking for placeholders, ambiguity, and scope creep. The user approves the written spec before any code exists.
21. Planning. The writing-plans skill activates only after the design is approved. It performs a scope check, designs the file structure, and decomposes the work into 2–5 minute tasks with explicit file paths and complete code blocks. The plan is saved next to the design.
22. Subagent-driven implementation. The subagent-driven-development skill dispatches a fresh Implementer subagent per task, with no session history inherited. The Implementer writes the failing test first, then the minimal implementation, then runs tests, then self-reviews.
23. Two-stage review. Once the Implementer reports complete, the Spec Reviewer subagent validates compliance line-by-line against the spec; it requires code inspection, not claims. Only after spec compliance passes does the Code Quality Reviewer subagent evaluate code quality, architecture, testing, and production readiness across four dimensions. If either reviewer rejects, the loop returns to the Implementer.
24. Finishing. The finishing-a-development-branch skill offers three options: open a PR, merge the branch locally, or stop. The user chooses.

6.3 Git worktree isolation

The using-git-worktrees skill activates automatically when the project is in a git repository. It creates a dedicated worktree per feature branch (`git worktree add feature/...`) so parallel subagents never clobber each other's files. Each worktree shares the single `.git` database with the main checkout but gets its own working tree; on completion, worktrees with no changes clean themselves up. This lets superpowers run what would otherwise be race-prone parallel work safely.

6.4 Concrete pipeline: a todo list app

Jesse Vincent's walkthrough of building a simple todo app with superpowers is the canonical example:

25. Claude loads the using-superpowers orientation skill and reads CLAUDE.md.
26. Brainstorm phase: it asks primary use case, scale, sync expectations; proposes three architecture options (local only, backend-optional, PWA with IndexedDB); the user chooses local-only; Claude writes `docs/superpowers/specs/2025-10-09-todo-app-design.md`.
27. Git worktree is created (`git worktree add feature/todo-app-v1`).
28. Planning phase: five tasks are written to `2025-10-09-todo-app-plan.md`, each with complete code blocks and shell commands.
29. Task 1: project scaffolding (Vitest, TypeScript). Implementer runs, commits "chore: initial project setup". Spec Reviewer passes. Code Quality Reviewer passes.
30. Task 2 (RED): Implementer writes the `addTodo` failing test, runs `npm test`, sees it fail, commits "test: red - add todo test". Reviewers pass.
31. Task 3 (GREEN): Implementer writes the minimal `addTodo` implementation, test goes green, commits "feat: green - implement addTodo". Reviewers pass.
32. Tasks 4 and 5: `listTodos`, RED then GREEN. Reviewers pass.
33. Finishing: Claude offers "open PR / merge / stop". User chooses PR. `gh pr create` runs.

6.5 How HealthCentral used superpowers

HealthCentral used superpowers for the SQLCipher encryption layer and the JWT auth module — exactly the places where test-first discipline was worth the overhead. Each cryptographic function was written against a failing test before a line of implementation existed. The two-stage review caught the one bug that mattered: a unique-IV generator that appeared correct in unit tests but whose entropy source was mis-imported and would have re-used IVs in production. The Spec Reviewer caught it; the Code Quality Reviewer would not have, because the unit tests were green. That single bug justified the entire setup cost.

7. Comparing the three frameworks

All three frameworks solve the same problem — keep the agent on task through SWE discipline — but with different architectures and trade-offs. The table below is a side-by-side view.

Dimension	superpowers	GSD v2	everything-claude-code
Packaging	Claude Code plugin (skills + marketplace)	Standalone TypeScript CLI on Pi SDK	Cross-platform config + plugin suite
Primary enforcement	Skill-based mandatory invocation	File-driven orchestrator + fresh-context agents	Skills + extensive PreToolUse / PostToolUse hooks
Planning artifact	docs/superpowers/specs/*-design.md + plan.md	PROJECT.md / REQUIREMENTS.md / ROADMAP.md / STATE.md / DECISIONS.md	AGENTS.md + per-skill SKILL.md
Research step	Explore + ask clarifying questions in brainstorm	4x parallel Researcher agents (stack, features, pitfalls, external)	Planner decomposes + docs-lookup skill
Review step	Spec Reviewer + Code Reviewer subagents (mandatory)	Plan Checker + Verifier + Debugger	code-reviewer + language reviewers + security-reviewer + AgentShield
Isolation	git worktree + fresh subagent context per task	Fresh 200K-token context per atomic task	Hook profiles (minimal / standard / strict) + subagent scoping
Parallelism	Limited (sequential subagents)	Dependency-aware waves of parallel workers	Orchestrator-coordinated subagents
Stuck-loop detection	Human spots it	Sliding-window pattern analyzer; one retry then halt	loop-operator agent + Stop hook
HITL gate location	After brainstorm (spec approval) + before merge	Milestone reconciliation + escalation on stuck-loop	Pre-commit + pre-push hooks + permission prompts

The right framework is the one whose enforcement strategy matches the team's risk model. For a solo developer who wants a disciplined workflow inside Claude Code, superpowers is the lightest weight and the easiest to adopt. For a team running long autonomous campaigns that need parallelism and observability, GSD v2 wins on orchestration. For an organization that wants defense in depth across multiple IDEs with strong security scanning, ECC is the most

complete package. In HealthCentral we used all three, because each was the right tool for a different part of the project.

8. Is this agile, waterfall, or something new?

The natural next question is what development methodology these frameworks actually embody. The honest answer is that none of them is pure agile and none is pure waterfall. They are closer to what Tony Alicea has called “Cascades” — rapid, verification-gated micro-waterfalls. Every slice follows the full sequence: specification, experimentation, implementation, verification, deployment. But because an agent can traverse that sequence in hours rather than the quarters a human team needs, you can afford the structure. Rigor stops being a tax and becomes a speed multiplier.

The agile comparison is useful but incomplete. Sprints map onto GSD slices; standups map onto STATE.md; retros map onto DECISIONS.md updates and the Stop-hook pattern extraction. The spirit of short cycles and continuous feedback is there. But the written spec is non-negotiable in a way that contradicts the agile manifesto’s “working software over comprehensive documentation.” In agentic development, the spec is the working software’s pre-condition — the agent literally cannot begin without it. That is waterfall’s discipline restored, compressed into a rapid cycle.

Put differently: the agile manifesto was written in 2001 for teams of humans who would misread a spec, argue about it, and benefit from frequent course-correction. Agentic systems do not benefit from that same ambiguity — they magnify it. They benefit from an exhaustive, precise specification they can execute against, with structured gates between stages. That is why the dominant pattern looks more waterfall than agile, and why it still ships faster than either pattern ever did alone.

9. Failure modes and how each framework answered them

Each of the four failure archetypes we opened with has a direct counter in the frameworks.

Premature action without grounding. Answered by spec-first gating in all three frameworks: superpowers will not let the brainstorming phase be skipped; GSD’s Plan Checker rejects incomplete plans; ECC’s planner agent produces the decomposition before any implementation subagent is dispatched.

Over-helpful substitution of missing entities. Answered by the two-stage review in superpowers and the Verifier in GSD. A hallucinated method name or fabricated import is caught because the Spec Reviewer or Verifier performs explicit code inspection against the spec, rather than trusting the Implementer’s narrative.

Context pollution from distractors. Answered by fresh-context spawning in all three. GSD makes this the central design principle — every task gets a new 200K window. Superpowers

uses the subagent-driven pattern for the same effect. ECC uses Stop-hook pattern extraction to externalize what must survive while letting the rest decay.

Fragile execution under load. Answered by the test harness. superpowers' Iron Law and ECC's tdd-guide enforce test-first; GSD's "truths" are automated checks that must pass before an artifact can ship. Every framework treats the test suite as the non-optional production floor.

HealthCentral's experience is instructive. Without these structural defenses, the first month of development was the usual mix of brilliant twenty-minute bursts and head-desk hours spent undoing work the model had convinced itself was correct. With the defenses in place — superpowers for cryptographic code, GSD for the long RAG evaluation campaign, ECC for cross-cutting security and HIPAA-adjacent concerns — the same four months shipped an actual local-first application that passes its own adversarial tests.

10. Reflections: rigid structure beats clever prompting

The clearest lesson from four months of working this way is that prompt cleverness has a hard ceiling and structure does not. A well-written prompt buys twenty minutes; a well-structured harness buys the rest of the project. Every attempt to compensate for missing structure with better prompting paid short-term interest and long-term debt. Every investment in structure — writing a CLAUDE.md, setting up a Verifier, running a two-stage review, locking a spec before coding — paid long-term dividends.

There is a deeper symmetry here. The frameworks that keep agents on task are recapitulating the advice experienced engineers give junior developers: write down what you intend to build; review your design with someone before coding; write a test before the code; have a second pair of eyes on the diff; make small atomic changes; refactor only on green. None of this is new. What is new is that the discipline can now be applied to a worker whose raw output speed is measured in tokens per second rather than lines per day. That is the real story of the last year. Senior engineering did not disappear. It moved into the harness.

For HealthCentral, this mattered because medical software does not get the luxury of a bug budget. Every line of code that touches patient data has to be right or refusedly absent. The frameworks above made it possible to approach that bar as a senior student, with a laptop, in one semester. That is not nothing.

11. References

- Evaluating Goal Drift in Language Model Agents (arXiv:2505.02709). <https://arxiv.org/abs/2505.02709>
- Anthropic alignment auditing research. <https://alignment.anthropic.com/2025/automated-auditing/>
- Effective Harnesses for Long-Running Agents (Anthropic). <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

- obra/superpowers repository. <https://github.com/obra/superpowers>
- Jesse Vincent, superpowers walkthrough. <https://blog.fsck.com/2025/10/09/superpowers/>
- gsd-build/gsd-2 repository. <https://github.com/gsd-build/gsd-2>
- gsd-build/get-shit-done repository (v1). <https://github.com/gsd-build/get-shit-done>
- Agent Native on Medium — "GET SH*T DONE: meta-prompting and spec-driven development." <https://agentnativdev.medium.com/get-sh-t-done-meta-prompting-and-spec-driven-development-for-claude-code-and-codex-d1cde082e103>
- affaan-m/everything-claude-code repository. <https://github.com/affaan-m/everything-claude-code>
- affaan-m/agentshield repository. <https://github.com/affaan-m/agentshield>
- Geoffrey Huntley, Ralph's loop. <https://ghuntley.com/ralph/>
- Tony Alicea, Not Agile. Not Waterfall. With AI It's Cascades. <https://tonyalicea.dev/blog/cascade-methodology/>
- Addy Osmani, The Code Agent Orchestra. <https://addyosmani.com/blog/code-agent-orchestra/>
- Addy Osmani, How to write a good spec for AI agents. <https://addyosmani.com/blog/good-spec/>
- LLM4TDD: Best Practices for Test Driven Development using LLMs (arXiv:2312.04687). <https://arxiv.org/abs/2312.04687>
- Claude Code hooks reference. <https://docs.claude.com/en/docs/claude-code/hooks>
- Anthropic — Equipping Agents for the Real World with Agent Skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>